

Annotations

Cosa sono i metadati e cosa sono le annotazioni

Definizione — Metadati. I metadati sono "informazioni sulle informazioni" (dati che descrivono altri dati). L'esempio usato: in una tabella di database, il dato è il valore `Sean Kennedy`; il **nome della colonna** (nome) è un metadato, perché ti dice *cosa* rappresenta quel valore — cioè che quella stringa è un nome.

Definizione — Annotazione. Un'annotazione è un modo per aggiungere metadati al codice. Ti permette di "attaccare" informazioni aggiuntive a classi, metodi, variabili, ecc. Il punto chiave è che le annotazioni sono **facoltative**: il codice compila anche senza. Ma **se le metti, devi usarle correttamente**, altrimenti il compilatore dà errore.

Concetto fondamentale (ripetuto più volte). Le annotazioni hanno origine dalle interfacce e funzionano in modo simile a loro. Contengono elementi che assomigliano a metodi astratti. La JVM, dietro le quinte, traduce gli elementi dell'annotazione in metodi di un'interfaccia, e l'annotazione stessa in un'implementazione di quell'interfaccia. Questo ti serve per capire perché la sintassi (che vedremo) somiglia tanto a quella di un'interfaccia.

Regola di stile. I nomi delle annotazioni sono *case-sensitive*, e per convenzione iniziano con la lettera maiuscola (come `@Override`).

Si applicano scrivendo il simbolo `@` seguito dal nome: `@Override`, `@FunctionalInterface`, ecc.

Annotazioni incorporate (built-in): `@Override` e `@FunctionalInterface`

Queste sono annotazioni che Java ti fornisce già pronte.

`@Override`

Definizione. Segnala che un metodo sta **sovrascrivendo** (override) un metodo del genitore. Ha due funzioni: comunica agli altri sviluppatori l'intenzione di fare override, e soprattutto fa sì che il **compilatore controlli** che tu stia davvero sovrascrivendo qualcosa.

Esempio della lezione (`toString`). C'è una classe `Person`. In Java ogni classe estende implicitamente `Object` (il compilatore inserisce `extends Object` dietro le quinte). `Object` ha già il metodo `toString()`.

- Se **non** fai l'override e stampi un oggetto `Person` con `System.out.println(...)`, ottieni l'output di default poco leggibile: nome del package + nome della classe + `@ + hashCode`.
- Facendo l'override di `toString()` per restituire, per esempio, il nome della persona, l'output diventa leggibile (`Sean Kennedy`).

Il vantaggio concreto di `@Override`. Se sbagli a scrivere il nome del metodo — per esempio scrivi `toString` con la *s* minuscola (`tostring`) — senza `@Override` Java penserebbe che è

semplicemente un *nuovo* metodo, e continuerebbe a usare quello di `Object` (output incomprensibile). Con `@Override`, il compilatore si accorge che quel nome non corrisponde a nessun metodo del genitore e ti dà **errore**. Questo protegge dagli errori di battitura.

Lo stesso vale quando `Person` implementa un'interfaccia con un metodo `move()`: annotando l'implementazione con `@Override` evidenzi che stai sovrascrivendo il metodo dell'interfaccia.

@FunctionalInterface

Definizione. Marca un'interfaccia come **interfaccia funzionale**, cioè un'interfaccia con **un solo metodo astratto** (Single Abstract Method). Il compilatore fa rispettare questa regola.

Esempio della lezione. Se un'interfaccia ha il metodo astratto `move()` ed è annotata con `@FunctionalInterface`, va bene. Se provi ad aggiungere un **secondo** metodo astratto, ottieni un errore.

Punto sottile importante. I metodi ereditati da `Object` **non contano**. Quindi se dichiari anche `equals()` nella tua interfaccia, questo *non* fa perdere all'interfaccia lo status di funzionale — perché `equals()` è un metodo di `Object` e la sua implementazione verrà comunque da lì. Continua a valere come interfaccia funzionale finché ha un solo metodo astratto "vero" (nell'esempio, `move()`).

Annotazioni per gli avvisi (warning) del compilatore: @Deprecated, @SuppressWarnings, @SafeVarargs

L'esempio usa una classe `Book` con tre metodi statici, richiamati dal `main` tramite metodi di istanza.

@Deprecated

Definizione. Marca un metodo (o altro elemento) come **deprecato**, cioè da non usare più — di solito perché esiste un'alternativa migliore. Non blocca la compilazione: genera solo un **avviso**.

Dettagli dalla lezione:

- Attenzione alla differenza tra l'annotazione `@Deprecated` (con la **D maiuscola**) e il tag Javadoc `@deprecated` (tutto **minuscolo**), che si mette nel commento di documentazione per spiegare *perché* è deprecato.
- L'annotazione può avere due proprietà: `since` (da quale versione è deprecato, es. "2.0") e `forRemoval` (`true/false`, se è destinato a essere rimosso in futuro).
- Gli editor mostrano il metodo deprecato **barrato** (con una lineetta sopra il nome).
- **Esempio reale dall'API Java:** il costruttore `Integer(...)` è deprecato in favore del metodo statico `Integer.valueOf(...)`, perché `valueOf` fa riferimento a un *pool* di interi e quindi **risparmia memoria**.
- Compilando con `javac`, l'avviso ti dice che il metodo `print` di `Book` è deprecato e contrassegnato per la rimozione. Non blocca il programma, è solo un avvertimento.

@SuppressWarnings

Definizione. Serve a dire al compilatore "**non mostrarmi questo avviso**". Si applica a un metodo (o elemento) specificando *quale* categoria di warning sopprimere.

Due usi mostrati nella lezione:

1. **@SuppressWarnings("deprecation")** — per sopprimere l'avviso di deprecazione. *Nota curiosa dal corso:* il docente mostra che in pratica, per il warning di *deprecazione*, l'avviso a volte **continua a comparire** lo stesso: ha verificato online (StackOverflow) e sembra un problema noto del compilatore non ancora risolto.
2. **@SuppressWarnings("unchecked")** — per sopprimere gli avvisi sui **tipi grezzi** (raw types). L'esempio: usare `new ArrayList<>()` con l'inferenza del tipo (l'operatore *diamond* `<>`) **non** dà avvisi. Invece usare un tipo grezzo — `new ArrayList()` senza parametro di tipo — genera un avviso di "operazioni non controllate o non sicure" (*unchecked or unsafe operations*). Applicando **@SuppressWarnings("unchecked")** sul metodo, l'avviso sparisce.

@SafeVarargs

Definizione. Si mette davanti a un metodo con **varargs** (argomenti variabili) per dichiarare che quel metodo si comporterà correttamente con gli argomenti che riceve, sopprimendo i relativi warning.

Esempio della lezione (un caso di uso scorretto, fatto apposta). Il docente crea un metodo chiamato apposta "abuse" perché **non** è realmente sicuro: riceve un elenco di interi come varargs, fa puntare un array di `Object` all'elenco di interi, poi fa puntare l'indice zero a una lista di stringhe → questo provoca una **ClassCastException**, perché si cerca di trattare una `String` come un `int`.

Regole importanti di @SafeVarargs (dette esplicitamente):

- Il metodo **deve** avere argomenti varargs (se togli i . . . non puoi usarla).
- Il metodo deve essere **static**, **private** oppure **final**. Un metodo **public** non-static/non-final dà problema; renderlo **private**, **final** o **static** risolve. (La ragione: questi modificatori garantiscono che il metodo non possa essere sovrascritto in modo insicuro.)

Creare annotazioni personalizzate (custom annotations)

Questo è il cuore della lezione. Finora si sono usate annotazioni già pronte; adesso si costruiscono le proprie.

La sintassi di base

Definizione. Un'annotazione personalizzata si definisce con **@interface** (la chiocciola *attaccata* alla parola `interface`):

```
public @interface MyAnnotation {  
}
```

- Può essere `public` o `package-private` (senza modificatore). Se la fai `public`, di solito devi metterla in un file separato. Nel corso il docente le tiene tutte in un unico file (non pubbliche) solo per comodità di spiegazione.

Annotazione marcatore (marker annotation)

Definizione. Un'annotazione marcatore è un'annotazione **senza elementi** — le parentesi graffe sono vuote. È il parallelo dell'**interfaccia marcatore** (una interfaccia senza metodi, come `Serializable`, `Cloneable`). Serve solo a "marcare" qualcosa, senza portare dati.

Esempio dalla lezione: `@interface Human { }` è un marker.

Elementi (o attributi) dell'annotazione

Definizione. Un **elemento** (detto anche attributo) è una proprietà dell'annotazione, che può contenere un valore. Sintatticamente **assomiglia a un metodo astratto** di un'interfaccia, ma non è un metodo: è un attributo con un tipo.

Esempio: dentro un'annotazione `Web`:

```
int startTime() default 9; // (riga ~12: "ora di inizio")
int hoursPerDay();        // (riga ~13: "ore al giorno")
```

Regole degli elementi (spiegate con i tentativi del docente):

- Come nelle interfacce, gli elementi sono **public e abstract di default**. Se provi a renderli `private`, `protected` o `final`, ottieni errore. (`final` e `abstract` sono incompatibili.)
- **default** rende un elemento **opzionale**: `int startTime() default 9;` significa che se non lo specifichi quando applichi l'annotazione, vale 9.
- Un elemento **senza default** diventa **obbligatorio**: devi passarlo per forza quando applichi l'annotazione, altrimenti errore ("*manca un valore per l'elemento richiesto*"). Nell'esempio, `hoursPerDay()` non ha default → è obbligatorio.

Costanti dentro le annotazioni. Puoi anche mettere delle costanti. Come nelle interfacce, sono `public static final` di default. Le costanti **non** contano come elementi — quindi anche un'annotazione marcatore (senza elementi) può contenere costanti.

Tipi ammessi per gli elementi

Definizione — regola sui tipi. Il tipo di un elemento può essere **solo** uno tra:

- un **tipo primitivo** (`int`, `boolean`, ecc.),
- **String**,
- un **enum**,
- un'altra **annotazione**,
- una **Class**,
- oppure un **array** di uno di questi.

NON sono ammessi i tipi wrapper (`Integer`, `Boolean`, ecc.). E sono ammessi **solo array monodimensionali**: un array 2D dà subito errore di "tipo non valido".

Esempi di elementi mostrati:

- `String name()` default `"..."`; (tipo `String`, con default per non renderlo obbligatorio)
- `Device consumes()` default `Device.LAPTOP`; (tipo enum; c'è un enum `Device { LAPTOP, PHONE }`)
- `Human human()` default `...`; (tipo annotazione: un'annotazione dentro un'altra)
- `Class<...> ...` (tipo `Class`)
- `String[] sites()`; (array di `String`)

Come si applica un'annotazione personalizzata

Con `@` + nome. Se ha elementi, li passi tra parentesi tonde nel formato `nomeElemento = valore`, separati da virgola, con il tipo corretto:

```
@Web(hoursPerDay = 6, startTime = 18)
```

```
@Human
```

```
class Student { }
```

- Se è un semplice marcatore, le parentesi non servono (ma puoi metterle vuote: `@Human()` va bene lo stesso).
- Puoi applicare più annotazioni allo stesso elemento, tutte su una riga o distribuite su più righe.
- Se passi un tipo sbagliato (es. una `String` dove serve un `int` per `hoursPerDay`) → errore.

Le scorciatoie (shorthand) nella sintassi delle annotazioni

Il compilatore ti offre delle abbreviazioni.

Scorciatoia 1 — l'elemento `value`

Definizione. Se applichi un'annotazione **senza specificare il nome dell'elemento** — es. `@Team("tennis")` invece di `@Team(value = "tennis")` — allora quel valore va automaticamente nell'elemento chiamato **`value`**.

Regola vincolante: perché questa scorciatoia funzioni, l'annotazione deve avere un elemento chiamato `value`, e **non può avere altri elementi obbligatori**. Se c'è un secondo elemento senza default (quindi obbligatorio), la scorciatoia smette di funzionare e ottieni errori, perché il compilatore non saprebbe a chi assegnare gli altri valori. Il fatto che `value` abbia o no un default è invece irrilevante.

Scorciatoia 2 — array con un solo elemento

Definizione. Quando passi a un elemento-array **un solo valore**, puoi **omettere le parentesi graffe**.

Esempio con `@Quiz(topics = {...})` dove `topics` è un `String[]`:

- Due o più valori: servono le graffe → `@Quiz(topics = {"conoscenza generale", "musica"})`

- Un solo valore: puoi scrivere `@Quiz(topics = "sport")` → il compilatore inserisce le graffe per te.

Cose che NON sono permesse:

- Con più valori, le graffe sono **obbligatorie** (`{"sport", "jazz"}`).
- **`null` non è ammesso** — serve un'espressione costante.
- Un array **vuoto** `{}` invece è accettato.

Scorciatoia 3 — combinare le due

Puoi unire "elemento `value`" + "array a singolo elemento". Con un'annotazione `@Colors` che ha un elemento `value` di tipo `String[]`, le quattro forme mostrate sono tutte equivalenti quando c'è un solo colore:

1. `@Colors(value = {"red"})` — tutto esplicito
2. `@Colors(value = "red")` — ometti le graffe (scorciatoia array)
3. `@Colors({"red"})` — ometti `value =` (scorciatoia `value`)
4. `@Colors("red")` — ometti **entrambi**

Annotazioni applicate ad altre annotazioni (meta-annotazioni)

Definizione. Fino a qui abbiamo applicato annotazioni a metodi, classi, campi, ecc. Adesso applichiamo annotazioni **alle annotazioni stesse**: queste si chiamano **meta-annotazioni**. Servono, per esempio, a limitare dove un'annotazione può essere usata, o a controllarne il ciclo di vita.

@Target

Definizione. Limita i **tipi di elementi** a cui la tua annotazione può essere applicata. Si specifica come un array di valori enum di tipo `ElementType`.

I valori di `ElementType` (a sinistra il valore, a destra dove agisce):

- `TYPE` → classi, interfacce, annotazioni
- `METHOD` → dichiarazioni di metodo
- `PARAMETER` → parametri di costruttore e di metodo
- `FIELD` → variabili di istanza e statiche
- `CONSTRUCTOR` → dichiarazioni di costruttore
- `LOCAL_VARIABLE` → variabili locali
- `ANNOTATION_TYPE` → altre annotazioni
- `TYPE_USE` → il più flessibile: **ovunque si usi un tipo** (creazione oggetti con `new`, cast, ecc.)

Dettaglio pratico. `java.lang` è importato automaticamente, ma il package delle annotazioni **no**: devi importare `ElementType` e `Target` da `java.lang.annotation`.

Esempio della lezione. Un'annotazione `@Data`:

- Senza `@Target` → si applica ovunque, nessun problema.

- Con `@Target({ElementType.FIELD, ElementType.PARAMETER})` → se provi ad applicarla a una classe, a un'interfaccia o a un costruttore ottieni: *"il tipo di annotazione non è applicabile a questo tipo di dichiarazione"*.
- Aggiungendo `CONSTRUCTOR` all'array, l'errore sul costruttore sparisce.
- Con `TYPE_USE` si può usare praticamente ovunque: variabile di istanza, variabile di classe, parametro di metodo, variabile locale, dentro un `new`, e persino in un `cast`. Tutti gli errori spariscono.

@Retention

Definizione. Controlla **fino a quando** l'annotazione viene conservata. Si collega al concetto di *type erasure* dei generics (dove certe informazioni vengono scartate dal compilatore quando il sorgente diventa file `.class`). Analogamente, le annotazioni possono essere scartate. Si specifica con un valore enum `RetentionPolicy`:

- **SOURCE** → l'annotazione esiste solo nel file sorgente; il compilatore la **scarta**, non finisce nel `.class`.
- **CLASS** → è memorizzata nel `.class` ma **non** è disponibile a runtime. È il **default**.
- **RUNTIME** → è memorizzata nel `.class` ed è disponibile a runtime tramite **reflection**.

Devi importare `Retention` e `RetentionPolicy` dal package `annotation`. Nell'esempio: un'annotazione con `@Retention(RetentionPolicy.SOURCE)` viene scartata e non è nel `.class`; un'altra con `@Retention(RetentionPolicy.RUNTIME)` resta disponibile a runtime.

@Inherited

Definizione. Rende un'annotazione **ereditabile**: se marchi una **classe genitore** con un'annotazione dichiarata `@Inherited`, quell'annotazione viene automaticamente ereditata dalle **sottoclassi**.

Esempio della lezione (@BreatheWater).

- C'è un'annotazione `@BreatheWater`. Una classe `Fish` è marcata con essa → "il pesce respira in acqua".
- Una classe `Trout` (trota) `extends Fish`. **Senza** `@Inherited`, `Trout` **non** eredita l'annotazione: per farla "respirare in acqua" dovresti annotarla esplicitamente.
- **Con** `@Inherited` sull'annotazione `@BreatheWater` (che va importata dal package `annotation`), basta annotare `Fish`: ogni sottotipo di `Fish` — inclusa `Trout` — eredita automaticamente l'annotazione, senza doverla ripetere.

@Repeatable

Definizione. Permette di applicare la **stessa annotazione più volte** sullo stesso elemento. Di default questo non è consentito. È utile quando vuoi usare la stessa annotazione con **valori diversi** ogni volta (quindi non ha senso per le annotazioni marcatore, che non hanno elementi).

Come funziona (richiede due annotazioni):

1. Un'annotazione **contenitore** (container), che ha un elemento `value` di tipo **array** dell'annotazione da ripetere. Per convenzione si chiama al **plurale**.

2. L'annotazione **ripetibile** vera e propria, marcata con `@Repeatable(Contenitore.class)`.

Il collegamento va in **entrambe le direzioni**: la ripetibile punta al contenitore (via `@Repeatable(...)`), e il contenitore punta alla ripetibile (via il tipo del suo array `value`).

Esempio della lezione (**Battery** / **Batteries**).

- Contenitore: `@interface Batteries { Battery[] value(); }`
- Ripetibile: `@Repeatable(Batteries.class) @interface Battery { String level(); boolean recharge(); }` — con due elementi: `level` (String) e `recharge` (boolean).
- Applicazione su una classe `ElectricCar`, **due volte**:
 - `@Battery(level = "high", recharge = false)` — se il livello è alto, non serve ricaricare.
 - `@Battery(level = "low", recharge = true)` — se il livello è basso, bisogna ricaricare.

Ecco come si ottiene un'annotazione applicabile più volte allo stesso tipo.